Facultad de Ingeniería y Arquitectura



MTM0120 · Álgebra Lineal y Métodos Numéricos

Grupo 1 · Equipo 5

Método de Regula Falsi en Python

Autores

Bismarck de Jesús Flores Delgado

Rommel Alexander Muñoz Guerrero

Alfredo Miguel Jarquín Castillo

Jeremy Josue Gradiz Noguera

Docente

Carlos Iván Argüello Martínez

Carrera

Ingeniería de Sistemas

Managua, Nicaragua, 10 de noviembre de 2026

1 Introducción y fundamento del método

Resolver una ecuación no lineal equivale a encontrar un valor de x para el cual f(x) = 0. El método de Regula Falsi, o falsa posición, aproxima ese valor mediante la recta que une los extremos de un intervalo con cambio de signo. En la calculadora se puede observar cómo cambia el intervalo y verificar la respuesta sustituyéndola en la función original.

Objetivo general

Integrar y comprobar un programa Python que resuelva ecuaciones no lineales por falsa posición dentro de la calculadora del curso.

Objetivos específicos

1. Implementar la interpolación de la secante y registrar las aproximaciones y sus medidas de parada.

2. Permitir la entrada de la función, la inspección gráfica y la elección de parámetros en la interfaz de escritorio.

3. Verificar la aproximación con una ecuación cúbica y contrastarla con el residuo y la tabla de iteraciones.

Condiciones de aplicación

Si f es continua en [a, b] y las evaluaciones f(a) y f(b) tienen signos opuestos, el teorema del valor intermedio garantiza al menos una raíz en el intervalo. Cuando uno de los extremos evalúa a cero, se devuelve ese extremo. La guía del curso presenta esta construcción como un método cerrado (Métodos numéricos, s. f., sección 1.2).

Deducción de la fórmula

La secante une (a, f(a)) y (b, f(b)). Su pendiente m es la diferencia de ordenadas dividida entre la diferencia de abscisas. El punto c es la abscisa donde la recta toma el valor cero.

$$m = [f(b) - f(a)] / (b - a)$$

$$0 = f(b) + m(c - b)$$

$$c = b - f(b)(b - a) / [f(b) - f(a)]$$

Si f(c) tiene signo opuesto a f(a), se sustituye b por c. Si f(c) tiene el mismo signo que f(a), se sustituye a por c. En ambos casos el subintervalo conserva el cambio de signo. Si f(c) evalúa a cero, termina el cálculo.

2 Implementación e integración

Algoritmo en Python

El módulo core/real_function.py interpreta la expresión y evalúa la función real. core/regula_falsi.py calcula el cruce de la secante. Para evitar una resta entre ordenadas grandes de signos contrarios, calcula la distancia desde el extremo más cercano al cruce mediante el cociente entre la magnitud menor y la mayor.

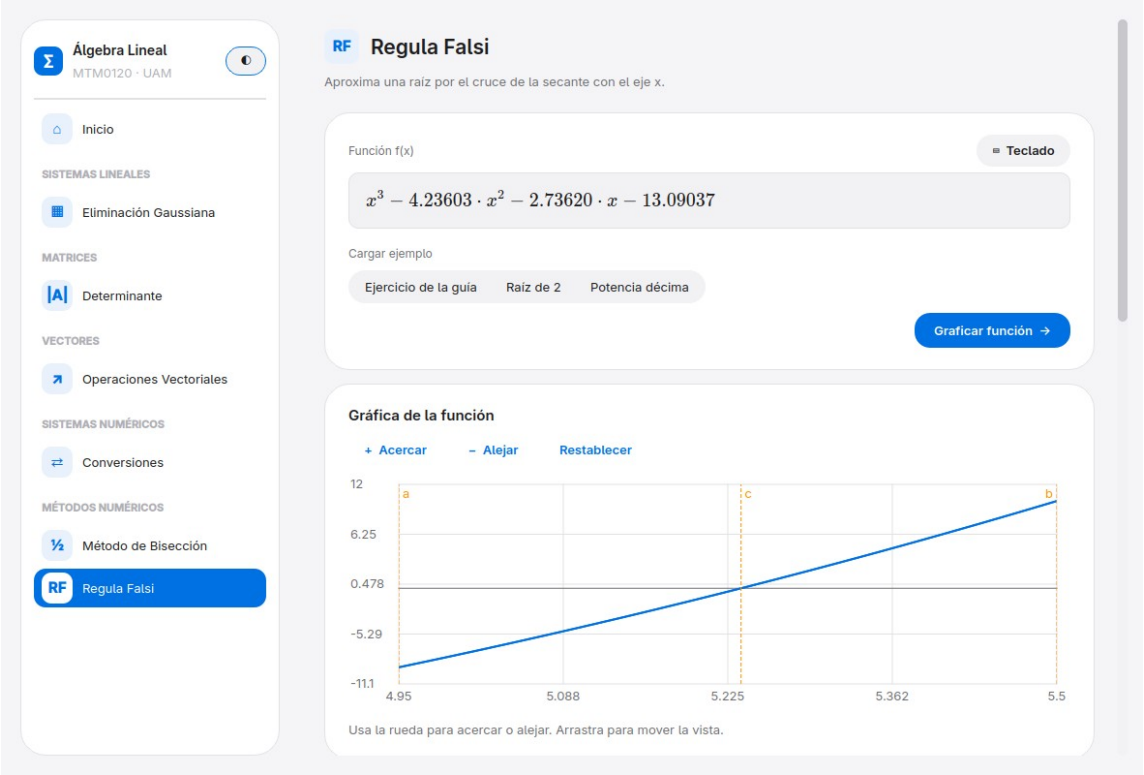
El recorrido compartido en core/root_bracket.py valida el orden de los extremos, la tolerancia positiva, el máximo de iteraciones y el cambio de signo. Cada fila guarda a, b, c, sus evaluaciones y el cambio relativo cuando es calculable. El resultado conserva la traza y distingue el cumplimiento de la tolerancia, el cero evaluado, el límite de iteraciones y el estancamiento por redondeo.

Uso en la calculadora

Después de instalar las dependencias de requirements-poc.txt, se ejecuta python revamp.py y se abre Métodos Numéricos → Regula Falsi. MathLive recibe la expresión dentro de Qt WebEngine. El formulario y la gráfica pertenecen a PySide6. Los recursos del editor se cargan localmente y la entrada de texto se puede activar si el editor visual falla.

Se introduce la función y se pulsa Graficar función. Después se ajustan a, b, la tolerancia, el criterio y el máximo de iteraciones. Calcular raíz ejecuta el motor Python y muestra la sustitución, la tabla y un selector para revisar cada iteración.

Figura 1. Editor MathLive y gráfica del polinomio en la calculadora.



3 Ejemplo y comprobación

Se seleccionó Ejercicio de la guía con el polinomio de la sección 1.2. El intervalo fue [4.95, 5.5], el máximo fue de 100 iteraciones y la tolerancia fue 0.01 % de cambio relativo.

$$f(x) = x^3 - 4.23603x^2 - 2.73620x - 13.09037$$

La aplicación obtuvo f(4.95) = −9.140510075 y f(5.5) = 10.0956225. Con esos valores, la primera secante cruza el eje x en c ≈ 5.2113457004. Como f(c) es negativa, c reemplaza al extremo inferior.

Tabla 1. Iteraciones registradas por la calculadora.

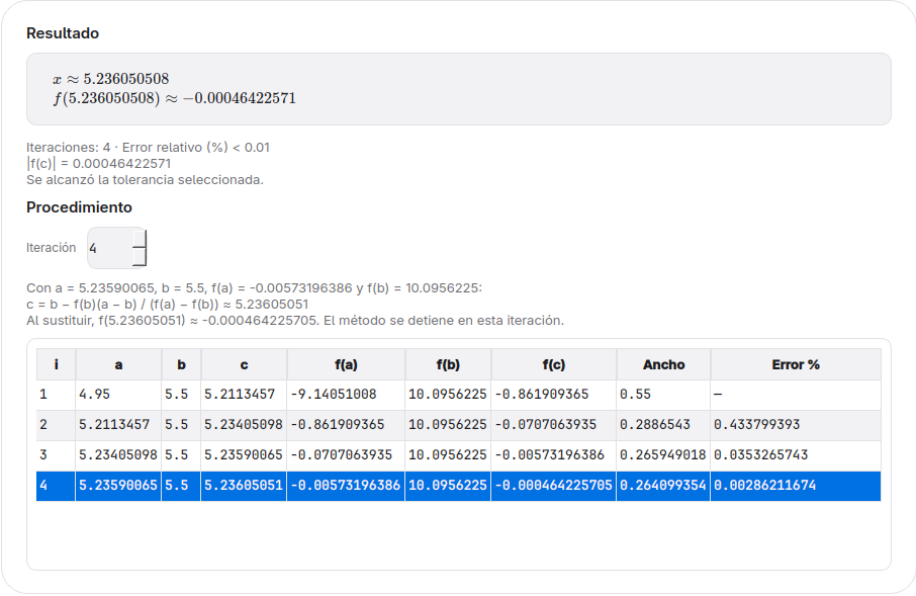
i	a	b	c	f(a)	f(b)	f(c)	Error %
1	4.9500000	5.5	5.2113457	−9.1405101	10.0956225	−0.8619094	—
2	5.2113457	5.5	5.2340510	−0.8619094	10.0956225	−0.0707064	0.4337994
3	5.2340510	5.5	5.2359006	−0.0707064	10.0956225	−0.0057320	0.0353266
4	5.2359006	5.5	5.2360505	−0.0057320	10.0956225	−0.0004642	0.0028621

Resultado

En la cuarta iteración, c ≈ 5.2360505080 y el cambio relativo fue 0.002862117 %, menor que 0.01 %. El programa se detuvo al cumplir el criterio seleccionado.

Al sustituir c en el polinomio se obtiene f(c) ≈ −0.0004642257. Su magnitud, 0.0004642257, es el residuo de la aproximación.

Figura 2. Resultado, sustitución y cuarta iteración seleccionada.



4 Análisis y conclusiones

Interpretación y limitaciones

En esta ejecución el extremo superior se mantuvo en 5.5. Las secantes produjeron aproximaciones sucesivas desde el lado negativo. El cambio relativo se redujo hasta satisfacer la tolerancia y el residuo permitió comprobar cuán cerca de cero quedó la función.

El cambio relativo mide la diferencia entre dos aproximaciones consecutivas. El residuo mide la distancia de f(c) a cero en el eje vertical. En la falsa posición clásica, un extremo puede permanecer fijo durante muchas iteraciones y el avance puede ser lento. La aplicación distingue el límite de iteraciones y la pérdida de avance por redondeo.

La continuidad debe revisarse en todo el intervalo. La gráfica ayuda a detectar problemas de dominio, pero una cantidad finita de muestras puede pasar por alto una discontinuidad. Las cifras de la tabla se obtuvieron evaluando el polinomio escrito en la guía, cuyas ordenadas impresas difieren de esa evaluación.

Conclusiones

La calculadora resuelve la ecuación no lineal mediante falsa posición en Python. La gráfica permite localizar el cruce y la traza permite revisar qué extremo se reemplazó. En el ejemplo, cuatro iteraciones bastaron para cumplir la tolerancia relativa.

La comprobación por sustitución y la indicación del motivo de parada permiten interpretar la aproximación. El mismo motor de intervalos comparte con Bisección la validación, los criterios de parada y el registro de iteraciones.

Referencias

Métodos numéricos: Búsqueda de raíces de ecuaciones. (s. f.). [Guía de ejercicios resueltos].

Flores, B. (s. f.). *linalg-calculator* [Código fuente]. GitHub. https://github.com/BismarckFlores/linalg-calculator